Practical : 5

Name: Payal Ganesh Ghanwat Roll No:222 Class: AIML(SE) Batch:S1

Problem Statement: Graph: Minimum Spanning Tree -- CO1, CO2, CO3

Represent a graph of your college campus using adjacency list /adjacency matrix. Nodes should represent the various departments/institutes and links should represent the distance between them.

Find minimum spanning tree :

a. Using Kruskal's algorithm.

b. Using Prim's algorithm.

CODE

```
INF = 999


# ---------------- Adjacency Matrix ---------------- #


graph = [
    [0, 2, 0, 6, 0],
    [2, 0, 3, 8, 5],
    [0, 3, 0, 0, 7],
    [6, 8, 0, 0, 9],
    [0, 5, 7, 9, 0]
]


departments = [
    "Computer",
    "IT",
    "Mechanical",
```

```
    "Civil",
    "Library"
]

V = len(graph)


# ================================================
#             PRIM'S ALGORITHM
# ================================================

def prim():
    selected = [False] * V
selected[0] = True

    edge = 0
cost = 0

    print("\nMinimum Spanning Tree using Prim's Algorithm\n")

    while edge < V - 1:
minimum = INF
        x = 0
y = 0

        for i in range(V):            if selected[i]:
for j in range(V):                    if not
selected[j] and graph[i][j]:                        if
```

```
graph[i][j] < minimum:
minimum = graph[i][j]
                        x = i
y = j

      print(
departments[x],
          "--",
          departments[y],
          "=",
graph[x][y]
      )

      cost += graph[x][y]
selected[y] = True        edge
+= 1

   print("Minimum Cost =", cost)



# ================================================
#            KRUSKAL'S ALGORITHM
# ================================================

parent = []



def find(i):
```

```
    while parent[i] != i:
i = parent[i]

    return i



def union(i, j):
    parent[find(i)] = find(j)



def kruskal():
edges = []

    # Create list of edges
for i in range(V):
        for j in range(i + 1, V):
if graph[i][j] != 0:
                edges.append((graph[i][j], i, j))

    # Sort edges according to weight
edges.sort()

    # Initialize parent array
parent.clear()

    for i in range(V):
parent.append(i)     cost = 0     count
= 0
```

```
    print("\nMinimum Spanning Tree using Kruskal's Algorithm\n")

    # Select edges
for w, u, v in edges:

        if find(u) != find(v):

            union(u, v)
             print(
departments[u],
                "--",
                departments[v],
                "=",
w
            )

            cost += w
count += 1

        if count == V - 1:
            break

    print("Minimum Cost =", cost)

# ================================================
#                 MAIN MENU
# ================================================
```

```
while True:

    print("\n1. Prim's Algorithm")
print("2. Kruskal's Algorithm")
print("3. Exit")

    choice = int(input("Enter Choice: "))

    if choice == 1:
        prim()

    elif choice == 2:
        kruskal()

    elif choice == 3:
        print("Program Ended")
        break

    else:
        print("Invalid Choice")
```

OUTPUT:

1. Prim's Algorithm
2. Kruskal's Algorithm
3. Exit

Enter Choice: 1

Minimum Spanning Tree using Prim's Algorithm

Computer -- IT = 2

IT -- Mechanical = 3

IT -- Library = 5

Computer -- Civil = 6

Minimum Cost = 16

Enter Choice: 2

Minimum Spanning Tree using Kruskal's Algorithm

Computer -- IT = 2

IT -- Mechanical = 3

IT -- Library = 5

Computer -- Civil = 6

Minimum Cost = 16